

1.A) arith.py

#program for Arithmetic operators

a=int(input("Enter a Number"))

b=int(input("Enter another Number"))

print("Arithmetic operations")

#addition

print('Sum: ',a+b)

#subtraction

print('Subtraction: ',a-b)

#multiplication

print ('Multiplication: ', a * b)

division

print('Division: ',a/b)

floor division

print('Floor Division: ',a//b)

modulo

print('Modulo: ',a%b)

a to the power b

print('Power: ',a**b)

print()

B). comp.py

#program for comparison operators

a=int(input("Enter a Number"))

b=int(input("Enter another Number"))

print("Comparison operations")

equal to operator

```
print('a == b =', a == b)
# not equal to operator
print('a != b =', a != b)
# greater than operator
print('a > b =', a > b)
# less than operator
print('a < b =', a < b)
# greater than or equal to operator
print('a >= b =', a >= b)
# less than or equal to operator
print('a <= b =', a <= b)
```

C) assign.py

```
#program for Assignment operators
print("Assignment operations")
a=int(input("Enter a Number"))
b=int(input("Enter another Number"))
print('a=b:', a==b)
print('a+=b:', a+b)
print('a-=b:', a-b)
print('a*=b:', a*b)
print('a%=b:', a%b)
print('a**=b:', a**b)
print('a//=b:', a//b)
```

D). bitwise.py

```
#program for Bitwise operators
a=int(input("Enter a Number"))
```

```
b=int(input("Enter another Number"))
print("Bitwise operations")
print ('a=',a,':',bin(a),'b=',b,':',bin(b))
c = 0
c = a & b;
print ("result of AND is ", c,':',bin(c))
c = a | b;
print ("result of OR is ", c,':',bin(c))
c = a ^ b;
print ("result of EXOR is ", c,':',bin(c))
c = ~a;
print ("result of COMPLEMENT is ", c,':',bin(c))
c = a << 2;
print ("result of LEFT SHIFT is ", c,':',bin(c))
c = a >> 2;
print ("result of RIGHT SHIFT is ", c,':',bin(c))
```

E) mem.py

```
a = int(input("Enter a number: "))
b = int(input("Enter another number: "))

print("Membership operations")

my_list = [1, 2, 3, 4, 5]
print("a:", a, "b:", b, "list:", my_list)
```

```
print("a is present in the given list" if a in my_list else "a is not present in the given list")
```

```
print("b is not present in the given list" if b not in my_list else "b is present in the given list")
```

```
c = int(b / a)
```

```
print("c:", c, "list:", my_list)
```

```
print("c is available in the given list" if c in my_list else "c is not available in the given list")
```

Results:

Thus the different operations using various types of operator python programs are executed successfully and verified.

Ex. No. 2 PYTHON PROGRAM USING CONTROL STRUCTURES

```
#!/usr/bin/python
```

```
# Control statements
```

```
total = 100
```

```
# country = "US"
```

```
country = "AU"
```

```
if country == "US":
```

```
    print("Shipping Country is United States")
```

```
if total <= 50:
```

```
    print("Shipping Cost is $75")
```

```
elif total <= 100:
    print("Shipping Cost is $50")
elif total <= 150:
    print("Shipping Cost is $25")
else:
    print("Shipping Cost is FREE")
```

```
if country == "AU":
    print("Shipping Country is Australia")
```

```
if total <= 50:
    print("Shipping Cost is $30")
elif total <= 100:
    print("Shipping Cost is $20")
elif total <= 150:
    print("Shipping Cost is $10")
else:
    print("Shipping Cost is FREE")
```

condi2.py

```
# Greeting for time of a day using conditional statements
```

```
time = int(input("Please enter the time: "))
```

```
if (time >= 600) and (time < 1200):
```

```
print("Morning")

elif time == 1200:
    print("Noon")

elif (time > 1200) and (time <= 1700):
    print("Afternoon")

elif (time > 1700) and (time <= 2000):
    print("Evening")

elif ((time > 2000) and (time < 2400)) or ((time >= 0) and (time < 600)):
    print("Night")

else:
    print("Invalid time!")
```

Result is success fully executed.

Ex. No.3 IMPLEMENTATION OF PYTHON PROGRAM USING FUNCTIONS

1. Builtin_func.py

```
#program using built-in functions
import math
n1 = int(input("Please enter a number : "))
exp = int(input("Please enter exponent value : "))
power = math.pow(n1,exp)
n2 = float(input("Please enter a float number : "))
```

```

print('The result of {} power {} = {}'.format(n1,exp,power))
#print the square root
print("The Square root of",n1,"is",math.sqrt(n1))
print("The factorial of", n1,"is: ", end="")
print(math.factorial(n1))
print ("The ceil valuuue of", n2,"is : ", end="")
print (math.ceil(n2))
print ("The floor value of", n2, "is : ", end="")
print (math.floor(n2))

```

2. user_func.py

Program to validate mobile number

```

def isphonenumner(numStr):
    l = len(numStr)

    if l < 10 or l > 13 or l == 11 or l == 12:
        return False

    for i in range(l):
        if l == 13 and i == 2:
            if numStr[i] != '-':
                return False
        else:
            if not numStr[i].isdigit():
                return False

    return True

```

```

ph_num = input("Enter a phone number: ")

```

```

if isphonenumner(ph_num):
    print("Valid phone number")
else:
    print("Invalid phone number")

```

3. recur_func.py

```

# Find factorial value using recursion function
def factorial(n):
    if n == 0:
        return 1
    else:
        return n * factorial(n - 1)

# Sum of digits using recursion function
def sum_digits(n):
    if n < 10:
        return n
    else:
        return n % 10 + sum_digits(n // 10)

n1 = int(input("Enter a number to find factorial: "))
n2 = int(input("Enter a positive number to find sum of digits: "))

f = factorial(n1)
s = sum_digits(n2)

print("The Factorial of", n1, "is", f)
print("The Sum of digit", n2, "is", s)

```

4. lamb_func.py

```

# Program using Lambda functions

# Lambda function to find the square value
sq = lambda n: n * n

# Lambda function to find the biggest number
big = lambda a, b: a if a > b else b

# Lambda function to add 15 to a number
radd = lambda a: a + 15

```

```

# Lambda function to multiply two numbers
rmul = lambda x, y: x * y

num = int(input("Enter a number to find the Square: "))
n1 = int(input("Enter First number: "))
n2 = int(input("Enter Second number: "))

print("The Square of", num, "is:", sq(num))
print("The Biggest value is:", big(n1, n2))
print("The First number is added", radd(n1))
print("The Multiplication of the given numbers are", rmul(n1, n2))

```

RESULT:

Thus python program using built-in functions, user defined functions, recursion functions and lambda functions are executed successfully and verified.

Ex. No.4 IMPLEMENTATION OF PYTHON PROGRAM USING STRINGS

1.alterstr.py

```

# Program for alternating the two strings

```

```
s1 = input("Enter First String: ")
s2 = input("Enter Second String: ")
```

```
if len(s1) == len(s2):
    print("String are with same length")
    result = "
```

```
    for i in range(len(s1)):
        result = result + (s2[i] + s1[i])
```

```
    print(result)
```

```
else:
```

```
    print("String are with different length")
```

```
2.search.py
```

```
s = input("Enter main string: ")
subs = input("Enter sub string: ")
```

```
flag = False
```

```
pos = -1
```

```
n = len(s)
```

```
c = 0
```

```
while True:
```

```
    pos = s.find(subs, pos + 1, n)
```

```
    if pos == -1:
```

```
        break
```

```
c = c + 1  
print("Found at position", pos)  
flag = True
```

```
if flag == False:  
    print("Not Found")  
else:  
    print("The number of occurrences:", c)
```

3.palind.py

Python program to check if a string is palindrome or not

First method

```
wo = input("Enter a String: ")  
w1 = ""
```

```
for i in wo:
```

```
    w1 = i + w1
```

```
if wo == w1:
```

```
    print("The given string is palindrome")
```

```
else:
```

```
    print("The given string is not palindrome")
```

Second method

```
string = input("Enter a word: ")
```

```
if string == string[::-1]:  
    print("The word is a palindrome")  
else:  
    print("The word is not a palindrome")
```

RESULT:

Thus the program using various string operations are executed successfully and verified.

Ex. No.5 IMPLEMENTATION OF PYTHON LIST USING BUILT-IN FUNCTIONS

1.empsallist.py

Program using built-in functions of Lists

```
n = int(input("Enter the number of employee: "))  
empname = []  
empsal = []
```

```
for i in range(n):  
    print("Enter", i + 1)  
    name = input("Employee Name: ")  
    num = int(input("Enter the Salary: "))  
    empname.append(name)  
    empsal.append(num)
```

printing maximum and minimum salary

```
manum = max(empsal)  
minum = min(empsal)
```

```
print("Maximum salary:", manum)
```

```
print("Minimum salary:", minum)
```

```
# updating the list
```

```
name = input("Enter the Employee name to add: ")
```

```
num = int(input("Enter the Salary to add: "))
```

```
empname.append(name)
```

```
empsal.append(num)
```

```
print("Updated Employee List:", empname)
```

```
print("Updated Employee Salary List:", empsal)
```

```
x = input("Enter the Employee Name to be removed: ")
```

```
print("The given name:", x)
```

```
if x in empname:
```

```
    empname.remove(x)
```

```
    print("Employee removed Successfully")
```

```
    print(empname)
```

```
else:
```

```
    print("Specified employee is not available")
```

```
2.orderlist.py
```

```
print(" ")
```

```
n = int(input("Enter the number of items in order1: "))
```

```
n1 = int(input("Enter the number of items in order2: "))
```

```
ord1 = [input("Enter Order-1 item: ") for _ in range(n)]  
ord2 = [input("Enter Order-2 item: ") for _ in range(n1)]
```

```
print("The Order1 and Order2 are equal" if ord1 == ord2 else "The order1  
and order2 are different")
```

```
ord3 = ord1.copy()  
ord3.extend(ord2)  
print("Order 3 elements are", ord3)
```

```
print("After sorting the Orders are")  
ord1.sort(); ord2.sort(); ord3.sort()  
print("Order1 is", ord1)  
print("Order2 is", ord2)  
print("Order3 is", ord3)
```

```
print("After reversing the Orders are")  
ord1.reverse(); ord2.reverse(); ord3.reverse()  
print("Order1 is", ord1)  
print("Order2 is", ord2)  
print("Order3 is", ord3)
```

```
l = len(ord3)  
print("The length of Order3 is", l)
```

```
it = input("Enter a item to find the index value: ")
```

```
print("Index position of given item in order3 is", ord3.index(it)) if it in ord3  
else print("Element not found")
```

```
name = input("Enter the item to add in the Order-3: ")  
ord3.insert(l, name)  
print("After inserting the element in Order3", ord3)
```

RESULT:

Thus the program implemented using various list built-in functions are executed successfully and verified.

Ex. No.6 IMPLEMENTATION OF PYTHON DICTIONARY

```
dictrec = {}  
n = int(input("Enter number of students: "))  
  
for _ in range(n):  
    dictrec[input("Enter Student Name: ")] = input("Enter % of Marks: ")  
  
print("Name of Student", "\t", "% of Marks")  
for x in dictrec:  
    print("\t", x, "\t", dictrec[x])  
  
dictrec[input("Enter student name to add: ")] = input("Enter % of Marks: ")  
print("Dictionary Elements are", dictrec)  
  
print("Length of Dictionary is", len(dictrec))
```

```
print("Dictionary Keys are", dictrec.keys())  
print("Dictionary Values are", dictrec.values())  
print("Dictionary Items are", dictrec.items())
```

```
dictrec1 = dictrec.copy()  
print("After removing")  
dictrec1.clear()  
print("Now Dictionary content", dictrec1)  
del dictrec1
```

```
dict1 = {'Ten': 10, 'Thirty': 30, 'Twenty': 20}  
dict2 = {'Fourty': 40, 'Sixty': 60, 'Fifty': 50}
```

```
dict3 = dict1.copy(); dict3.update(dict2)
```

```
print("Dictionary 1:", dict1)  
print("Dictionary 2:", dict2)  
print("Merged Dictionary:", dict3)
```

```
print("The Dictionary are equal" if dict1 == dict2 else "The Dictionary are  
not equal")
```

```
print("Sorting Dictionary using Keys")  
print(sorted(dict3.keys()))  
print("Sorting Dictionary using Values")  
print(sorted(dict3.values()))  
print("Sorting Dictionary using Items")
```

```
print(sorted(dict3.items()))
```

RESULT:

Thus the program implemented using python dictionaries are executed successfully and verified.

Ex. No.7 IMPLEMENTATION OF PYTHON TUPLES

```
fruit = ('apple', 'banana', 'cherry', 'mango', 'grape', 'orange')
```

```
print("\nCreated tuple is :", fruit)
```

```
print("\nSecond fruit is :", fruit[1])
```

```
print("\nFrom 3-6 fruits are :", fruit[3:6])
```

```
print("\nList of all items in Tuple :")
```

```
[print(x) for x in fruit]
```

```
fru = input("Enter the fruit name to find in the tuple:")
```

```
if fru in fruit: print("\nYes,", fru, "is in the fruits tuple")
```

```
print("\nLength of Tuple is :", len(fruit))
```

```
tup1, tup2 = (1, 2, 3, 4, 5), (6, 7, 8, 9, 10)
```

```
print("Tuple1 element are:", tup1)
```

```
print("Tuple2 element are:", tup2)
```

```
tup3 = tup1 + tup2
```

```
print("Concatenated Tuple:", tup3)
```

```
print("Tuple repetition:", tup1 * 3)
```

```
print("Maximum Element in Tuple1:", max(tup1))
```

```
print("Minimum Element in Tuple2:", min(tup2))
```

```
print("Length of Tuple:", len(tup3))
```

```
print("Tuple from List:", tuple([11, 12, 13, 14, 15]))
```

```
print("Count of Element 2:", tup1.count(2))
```

```
print("Index of Element 7:", tup2.index(7))
```

```
test_tup = ([7, 5, 4], [8, 2, 4], [0, 7, 5])
```

```
print("The original tuple is :", test_tup)
```

```
print("The tuple after sorting lists :", tuple(sorted(x) for x in test_tup))
```

RESULT:

Thus the program implemented using python tuples are executed successfully and verified.

Ex. No.8 IMPLEMENTATION OF PYTHON FILE HANDLING

```
import pickle, tempfile
```

```
print("WORKING WITH BINARY FILES")
```

```
print("Enter Records of Employees\n")
```

```
f = tempfile.NamedTemporaryFile(delete=False).name
```

```
r = 1
```

```
with open(f, "wb") as b:
```

```
while True:
    print("RECORD No.", r)
    e = int(input("\tEmployee number : "))
    n = input("\tEmployee Name : ")
    bs = int(input("\tBasic Salary : "))
    al = int(input("\tAllowances : "))
    t = bs + al
    print("\tTOTAL SALARY : ", t)

    pickle.dump([e, n, bs, al, t], b)
    r += 1
    if input("Do you wish to enter more records (y/n)? ").lower() == 'n':
        print("Record entry OVER\n")
        break

print("Size of binary file (in bytes):", b.tell())

print("Now reading the employee records from the file\n")

i = 1
try:
    with open(f, "rb") as b:
        while True:
            print("Record Number :", i)
            print(pickle.load(b))
            i += 1
```

```
except EOFError:
```

```
    pass
```

RESULT:

Thus the file program implemented using employee details are executed successfully and verified.

Ex. No.9

MANIPULATION OF DATE AND TIME

```
from datetime import date
```

```
import datetime, calendar, time
```

```
y1,m1,d1 = int(input("Enter First Date Year:")), int(input("Enter First Date  
Month:")), int(input("Enter First Date Day:"))
```

```
y2,m2,d2 = int(input("Enter Second Date Year:")), int(input("Enter Second  
Date Month:")), int(input("Enter Second Date Day:"))
```

```
d_1, d_2 = date(y1,m1,d1), date(y2,m2,d2)
```

```
print("The difference between Date2-Date1:", d_2 - d_1)
```

```
now = datetime.datetime.now()
```

```
today = date.today()
```

```
print("Now Date and Time is:", now)
```

```
print("Current Date is:", today)
```

```
print("Current year:", today.year)
```

```
print("Current month:", today.month)
```

```
print("Current day:", today.day)
```

```
t = time.localtime()
```

```
print("Hours:", t.tm_hour)
```

```
print("Minutes:", t.tm_min)
```

```
print("Seconds:", t.tm_sec)
```

```
print("Current Time:", now.time())
```

```
print("Current Time:", time.ctime())
```

```
print("date1 =", today.strftime('%d/%m/%Y'))
```

```
print("date2 =", today.strftime('%B %d, %Y'))
```

```
print("date3 =", today.strftime('%m/%d/%Y'))
```

```
print("date4 =", today.strftime('%b-%d-%Y'))
```

```
print("Here is the Date1 Month calendar:")
```

```
print(calendar.month(y1, m1))
```

```
print("Here is the Date2 Month calendar:")
```

```
print(calendar.month(y2, m2))
```

```
print("Here is the Date1 calendar:")
```

```
print(calendar.calendar(y1))
```

```
print("Here is the Date2 calendar:")
```

```
print(calendar.calendar(y2))
```

RESULT:

Thus the manipulation of date and time using datetime module and calendar modules are executed successfully and verified.

Ex. No.10

IMPLEMENTATION OF EXCEPTION HANDLING

Excepnum.py

```
def div(a, b):
```

```
    try: print(f"The result of {a} divided by {b} is: {a/b}")
```

```
    except ZeroDivisionError: print("Error: Cannot divide by zero!")
```

```
    except TypeError: print("Unsupported Data type")
```

```
    finally: print("This is finally block")
```

```
def add(a, b):
```

```
    try: print(f"The result of {a} added with {b} is: {a+b}")
```

```
    except TypeError: print("Unsupported Data type")
```

```
    finally: print("The Finally block")
```

```
n1, n2 = int(input("Enter Value1: ")), int(input("Enter Value2: "))
```

```
s1, s2 = input("Enter String1: "), input("Enter String2: ")
```

```
print("Division operation", n1, n2); div(n1, n2)
```

```
print("\nDivision operation", n1, "34"); div(n1, "34")
```

```
print("\nString Addition", s1, s2); add(s1, s2)
```

```
print("\nString Addition", s1, 78); add(s1, 78)
```

2. exceptfile.py

Error (waiting)

RESULT:

Thus the exception handling implemented for arithmetic operations and file operations are executed successfully and verified.